Lab Assignment: Dynamic User Directory & Live Clock

---

1. Lab Overview & Objectives

In this lab, you will build an interactive User Management Dashboard from scratch. The assignment bridges foundational HTML/CSS structure with modern JavaScript features: DOM manipulation, ES6 Classes, object methods, asynchronous timers, simulated network Promises, and async/await flow control.

Covered Competencies

- **HTML5:** Forms, text/select inputs, semantic tables (thead, tbody, tr, th, td).
- **CSS:** External stylesheets, class and ID selectors, tabular formatting, and feedback styles.
- **JavaScript Fundamentals:** DOM selection, innerHTML, event handling, and Date object.
- **ES6 OOP:** Class declaration, constructor methods, and instance methods.
- **Asynchronous Flow:** setInterval timers, Promise constructors with resolve/reject, and async/await syntax.

2. Workspace & File Structure

Create a single working folder containing exactly the following three files:

- **index.html:** Main HTML document structuring the layout, clock display, input form, and results table.
- **style.css:** External CSS stylesheet linked in the HTML head section.
- **app.js:** External JavaScript file embedded at the bottom of the HTML body.

3. Detailed Tasks Specification

Task 1: HTML Structure (index.html)

- **Linking Files:** Link style.css inside <head> and include app.js right before </body>.
- **Live Clock Container:** Create a prominent container with an <h2> having id="clock".
- **User Registration Form:** Build a form with id="user-form" containing:
  - Text input for Full Name with name="fullName" and an associated label.
  - Text input for Email Address with name="email" and an associated label.
  - Drop-down selection (<select name="role">) with options: Admin, Editor, and Viewer.
  - Submit button (<input type="submit" value="Add User"> or <button>).
- **Status Feedback:** Add an empty paragraph element to show success/error alerts.
- **User Records Table:** Create a table with id="user-table", including:
  - A <thead> row containing headers: Name, Email, Role, and Registered At.
  - An empty <tbody> with id="user-table-body".

Task 2: Interface Styling (style.css)

- **General Page Layout:** Apply a clean font-family (e.g., Arial, sans-serif) to body, center the main content container, and set a clean background color.
- **Digital Clock:** Style #clock with bold font weight, large size (e.g., 24px), centered text, and distinct color.
- **Table Design:** Set table width to 80%, collapse borders (border-collapse: collapse), and add borders to cells.

- **Header Contrast:** Apply a dark background color to <th> elements with bold, white text.
- **Feedback Utility Classes:** Define .success (green color, bold) and .error (red color, bold).

Task 3: Real-Time Clock with setInterval (app.js)

- **updateClock() Function:** Instantiate a new Date() object and retrieve hours, minutes, and seconds using getHours(), getMinutes(), and getSeconds().
- **Time Formatting:** Format the string as HH:MM:SS (padding single digits with leading zeros where applicable).
- **DOM Injection:** Inject the formatted string into the #clock element.
- **Interval Registration:** Set an interval to invoke updateClock every 1000 milliseconds (1 second).

Task 4: Object-Oriented Modeling with ES6 Classes (app.js)

- **Class User:** Define an ES6 class named User.
- **Constructor:** Accept name, email, and role. Initialize instance properties: this.name, this.email, this.role, and this.createdAt = new Date().
- **toTableRow() Method:** Add a method returning a table row template string: <tr><td>...</td></tr> displaying the user details along with this.createdAt.toLocaleTimeString().

Task 5: Simulated Asynchronous Service (app.js)

- **saveUserAsync(user):** Create a function returning a new Promise((resolve, reject) => { ... }).
- **Latency Simulation:** Use setTimeout to enforce an artificial 1.5-second (1500 ms) delay.
- **Validation Logic:** If user.name.trim().length < 3, reject with: "Error: Name must be at least 3 characters long.".
- **Fulfillment:** Otherwise, resolve with an object: { success: true, data: user }.

Task 6: Event Handling & Async/Await Integration (app.js)

- **Submit Event Listener:** Attach an event listener to #user-form for the submit event.
- **Default Cancellation:** Use event.preventDefault() to suppress page reload.
- **Async Handler Workflow:** Implement an async function that:
  - Reads the values of fullName, email, and role from the form.
  - Instantiates a new User object.
  - Sets #status-message text to "Saving user to server..." with neutral styling.
  - Uses try...catch to await saveUserAsync(newUser).
  - Inside try: appends newUser.toTableRow() into #user-table-body, clears form inputs, sets status to "User registered successfully!", and adds .success class.
  - Inside catch(err): displays the error message in #status-message and adds .error class.